Deep Network Development

Autumn 2026, Practice #7

Viktor Varga
Department of Artificial Intelligence, ELTE IK

Previously - NumPy ndarray type attributes

The `numpy.ndarray` type

```
temps2d = np.array([[-0.5, -1.5, 5.9, 12.9, 18.1, 21.6, 23.1, 24.2, 15.0, 11.7, 3.7, -3.6],
           [-4.3, -3.5, 3.1, 13.5, 17.4, 25.1, 28.3, 24.3, 18.5, 13.1, 0.3, -4.4]], dtype=np.float32)
```

- **`temps2d.size`** **`24`**
- **`temps2d.ndim`** **`2`**
- **`temps2d.shape`** **`(2, 12)`**
- **`temps2d.dtype`** **`float32`**
- **`temps2d.itemsize`** **`4`**
- **`temps2d.strides`** **`(48, 4)`**
- **`temps2d.base`** **`None`**

Software for neural networks













CPU only

CPU & GPU

PyTorch - The Tensor type

The `torch.Tensor` type

```
temps2d = torch.tensor([[-0.5, -1.5, 5.9, 12.9, 18.1, 21.6, 23.1, 24.2, 15.0, 11.7, 3.7, -3.6],
        [-4.3, -3.5, 3.1, 13.5, 17.4, 25.1, 28.3, 24.3, 18.5, 13.1, 0.3, -4.4]], dtype=torch.float32)
```

- `temps2d.numel()` `24`
- `temps2d.ndim` `2`
- `temps2d.size()` `torch.Size([2, 12])`
- `temps2d.shape` `torch.Size([2, 12])`
- `temps2d.dtype` `torch.float32`
- `temps2d.itemsize` `4`
- `temps2d.stride()` `(12, 1)`
- `temps2d._base` `None`
- `temps2d.device` `device(type='cpu')`
- `temps2d.requires_grad` `False`
- `temps2d.grad` `None`
- `temps2d.grad_fn` `None`

PyTorch - The Tensor type

The `torch.Tensor` type

```
temps2d = torch.tensor([[-0.5, -1.5, 5.9, 12.9, 18.1, 21.6, 23.1, 24.2, 15.0, 11.7, 3.7, -3.6],
        [-4.3, -3.5, 3.1, 13.5, 17.4, 25.1, 28.3, 24.3, 18.5, 13.1, 0.3, -4.4]], dtype=torch.float32)
```

-	`temps2d.numel()`	`24`	∈ `ndarray.size`
-	`temps2d.ndim`	`2`	∈ `ndarray.ndim`
-	`temps2d.size()`	`torch.Size([2, 12])`	∈ `ndarray.shape`
-	`temps2d.shape`	`torch.Size([2, 12])`	∈ `ndarray.shape`
-	`temps2d.dtype`	`torch.float32`	∈ `ndarray.dtype`
-	`temps2d.itemsize`	`4`	∈ `ndarray.itemsize`
-	`temps2d.stride()`	`(12, 1)`	∈ `ndarray.strides`
-	`temps2d._base`	`None`	∈ `ndarray.base`
-	`temps2d.device`	`device(type='cpu')`	
-	`temps2d.requires_grad`	`False`	
-	`temps2d.grad`	`None`	
-	`temps2d.grad_fn`	`None`	

Slightly different naming conventions…

While `Tensor.shape` is not a tuple, but nonetheless:

`( temps2d.shape == (2, 12) ) is True`

PyTorch - The Tensor type

The `torch.Tensor` type

```
temps2d = torch.tensor([[-0.5, -1.5, 5.9, 12.9, 18.1, 21.6, 23.1, 24.2, 15.0, 11.7, 3.7, -3.6],
        [-4.3, -3.5, 3.1, 13.5, 17.4, 25.1, 28.3, 24.3, 18.5, 13.1, 0.3, -4.4]], dtype=torch.float32)
```

-	`temps2d.numel()`	`24`	∈	`ndarray.size`
-	`temps2d.ndim`	`2`	∈	`ndarray.ndim`
-	`temps2d.size()`	`torch.Size([2, 12])`	∈	`ndarray.shape`
-	`temps2d.shape`	`torch.Size([2, 12])`	∈	`ndarray.shape`
-	`temps2d.dtype`	`torch.float32`	∈	`ndarray.dtype`
-	`temps2d.itemsize`	`4`	∈	`ndarray.itemsize`
-	`temps2d.stride()`	`(12, 1)`	∈	`ndarray.strides`
-	`temps2d._base`	`None`	∈	`ndarray.base`
-	`temps2d.device`	`device(type='cpu')`		
-	`temps2d.requires_grad`	`False`		
-	`temps2d.grad`	`None`		
-	`temps2d.grad_fn`	`None`		

Strides are not given in bytes, but in number of elements.

PyTorch - Features

The features of NumPy are mostly present in PyTorch too.

```
temps2d = torch.tensor([[-0.5, -1.5, 5.9, 12.9, 18.1, 21.6, 23.1, 24.2, 15.0, 11.7, 3.7, -3.6],
        [-4.3, -3.5, 3.1, 13.5, 17.4, 25.1, 28.3, 24.3, 18.5, 13.1, 0.3, -4.4]], dtype=torch.float32)
```

```
summer_months = temps2d[:, 5:8]
```

An example for basic indexing in PyTorch.

```
some_months = temps2d[:, [1, 4, 9]]
```

An example for advanced indexing in PyTorch.

PyTorch - Features

The features of NumPy are mostly present in PyTorch too.

```
temps2d = torch.tensor([[-0.5, -1.5, 5.9, 12.9, 18.1, 21.6, 23.1, 24.2, 15.0, 11.7, 3.7, -3.6],
        [-4.3, -3.5, 3.1, 13.5, 17.4, 25.1, 28.3, 24.3, 18.5, 13.1, 0.3, -4.4]], dtype=torch.float32)
```

An example for a PyTorch collapse operation

```
yearly_avg_temps = torch.mean(temps2d, axis=1)    # (2, 12) -> (2,)
```

An example for a PyTorch elementwise op and broadcasting.

```
diffs = temps2d - yearly_avg_temps[:, None]       # (2, 12) v (2, 1)
```

PyTorch - Features

Some differences are present between the two APIs…

A few examples:

```
temps2d[:, ::-1]          # -> "ValueError: step must be greater than zero"
```

```
temps2d_reversed = torch.flip(temps2d, dims=(1,))
```

PyTorch - Features

Some differences are present between the two APIs…

A few examples:

```
temps2d[:, ::-1]          # -> "ValueError: step must be greater than zero"
```



Negative strides are not implemented in PyTorch.
Reversing the order of indices along an axis is not possible **by creating a view.**

```
temps2d_reversed = torch.flip(temps2d, dims=(1,))
```

`torch.flip()` can be used, but while `np.flip()`
creates a view in NumPy, this creates a new tensor in PyTorch…

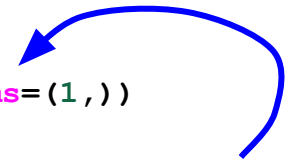
PyTorch - Features

Some differences are present between the two APIs…

A few examples:

```
temps2d[:, ::-1]          # -> "ValueError: step must be greater than zero"
```

```
temps2d_reversed = torch.flip(temps2d, dims=(1,))
```



Occasionally, different naming conventions are used
(mostly originating from the classic Lua Torch library).

NumPy array → PyTorch tensor conversion

```
my_array = np.arange(6, dtype=np.float32)

my_tensor1 = torch.tensor(my_array)
print( type(my_tensor1) )                  # <class 'torch.Tensor'>
print( my_tensor1.shape )                  # torch.Size([6])
print( my_tensor1.dtype )                  # torch.float32
print( my_tensor1 )                        # tensor([0., 1., 2., 3., 4., 5.])

my_tensor2 = torch.from_numpy(my_array)
print( type(my_tensor2) )                  # <class 'torch.Tensor'>
print( my_tensor2.shape )                  # torch.Size([6])
print( my_tensor2.dtype )                  # torch.float32
print( my_tensor2 )                        # tensor([0., 1., 2., 3., 4., 5.])

my_tensor1[2] = 42
my_tensor2[3] = 99

print(my_array)                            # [ 0.  1.  2. 99.  4.  5.]
```

NumPy array → PyTorch tensor conversion

```
my_array = np.arange(6, dtype=np.float32)

my_tensor1 = torch.tensor(my_array)
print( type(my_tensor1) )                  # <class 'torch.Tensor'>
print( my_tensor1.shape )                  # torch.Size([6])
print( my_tensor1.dtype )                  # torch.float32
print( my_tensor1 )                        # tensor([0., 1., 2., 3., 4., 5.])

my_tensor2 = torch.from_numpy(my_array)
print( type(my_tensor2) )                  # <class 'torch.Tensor'>
print( my_tensor2.shape )                  # torch.Size([6])
print( my_tensor2.dtype )                  # torch.float32
print( my_tensor2 )                        # tensor([0., 1., 2., 3., 4., 5.])

my_tensor1[2] = 42
my_tensor2[3] = 99

print(my_array)                            # [ 0.  1.  2. 99.  4.  5.]
```

`torch.tensor()`:
Creates a new tensor by **copying** the contents of the array (or list).

`torch.from_numpy()`:
Creates a new tensor **that shares the underlying memory*** with the array.

* Except in certain cases

PyTorch tensor → NumPy array conversion

```
my_tensor = torch.arange(6, dtype=torch.float32)

my_array = my_tensor.numpy()

print( type(my_array) )                    # <class 'numpy.ndarray'>
print( my_array.shape )                    # (6,)
print( my_array.dtype )                    # float32
print( my_array )                          # [0. 1. 2. 3. 4. 5.]


my_array[2] = 42

print( my_array )                           # [ 0.  1. 42.  3.  4.  5.]
print( my_tensor )                          #  tensor([ 0.,  1., 42.,  3.,  4.,  5.])
```

`torch.Tensor.numpy()`:
Creates a new array
that shares the underlying memory* with the tensor.

* **Except in certain cases**

PyTorch - `torch.nn.Module`

```
class BinClassifierMLP(nn.Module):

  def __init__(self, n_input_vars):
        super().__init__()
        self.linear1 = nn.Linear(n_input_vars, 30)
        self.linear2 = nn.Linear(30, 20)
        self.linear3 = nn.Linear(20, 1)

  def forward(self, x):
      h = self.linear1(x)
      h = nn.functional.relu(h)
      h = self.linear2(h)
      h = nn.functional.relu(h)
      h = self.linear3(h)
      h = nn.functional.sigmoid(h)
      return h
```

`torch.nn.Module:` A container type for the representation of neural networks or their components.
When we want to define a new neural network or a component, **we should create a new subclass.**

PyTorch - `torch.nn.Module`

```
class BinClassifierMLP(nn.Module):

  def __init__(self, n_input_vars):
        super().__init__()
        self.linear1 = nn.Linear(n_input_vars, 30)
        self.linear2 = nn.Linear(30, 20)
        self.linear3 = nn.Linear(20, 1)

  def forward(self, x):
      h = self.linear1(x)
      h = nn.functional.relu(h)
      h = self.linear2(h)
      h = nn.functional.relu(h)
      h = self.linear3(h)
      h = nn.functional.sigmoid(h)
      return h
```

`torch.nn.Linear:` A built-in class representing fully-connected layers. **`Linear`** is also a subclass of **`torch.nn.Module`** itself.

This way, neural networks are usually defined as a hierarchical structure of **`torch.nn.Module`** objects.

`torch.nn.Module:` A container type for the representation of neural networks or their components.
When we want to define a new neural network or a component, **we should create a new subclass.**

PyTorch - `torch.nn.Module`

```
class BinClassifierMLP(nn.Module):

  def __init__(self, n_input_vars):
        super().__init__()
        self.linear1 = nn.Linear(n_input_vars, 30)
        self.linear2 = nn.Linear(30, 20)
        self.linear3 = nn.Linear(20, 1)

  def forward(self, x):
      h = self.linear1(x)
      h = nn.functional.relu(h)
      h = self.linear2(h)
      h = nn.functional.relu(h)
      h = self.linear3(h)
      h = nn.functional.sigmoid(h)
      return h
```

`__init__():`
In the initialization method we should specify the stateful components of our network.

All member fields (self.*) that are **`torch.nn.Module`** instances will be registered by PyTorch and if they have parameters, they can be trained.

`forward(inputs):`
The hypothesis function implementation. If called with the inputs of the network, it should compute and return the outputs (usually the prediction) of the network.

PyTorch - `torch.nn.Module`

```
class BinClassifierMLP(nn.Module):

  def __init__(self, n_input_vars):
        super().__init__()
        self.linear1 = nn.Linear(n_input_vars, 30)
        self.linear2 = nn.Linear(30, 20)
        self.linear3 = nn.Linear(20, 1)

  def forward(self, x):
      h = self.linear1(x)
      h = nn.functional.relu(h)
      h = self.linear2(h)
      h = nn.functional.relu(h)
      h = self.linear3(h)
      h = nn.functional.sigmoid(h)
      return h
```

```
bincl_model = BinClassifierMLP( n_input_vars = inputs_train.shape[1] )
train_batch_ys_pred = bincl_model(train_batch_xs)
```

PyTorch - `torch.nn.Module`

```
class BinClassifierMLP(nn.Module):

  def __init__(self, n_input_vars):
        super().__init__()
        self.linear1 = nn.Linear(n_input_vars, 30)
        self.linear2 = nn.Linear(30, 20)
        self.linear3 = nn.Linear(20, 1)

  def forward(self, x):
      h = self.linear1(x)
      h = nn.functional.relu(h)
      h = self.linear2(h)
      h = nn.functional.relu(h)
      h = self.linear3(h)
      h = nn.functional.sigmoid(h)
      return h
```

`torch.nn.Module` by default, implements **`__call__():`**
Due to this, we can evaluate the hypothesis function, simply, by calling the instance of our Module as a function with the input tensors.

`torch.nn.Module( inputs )` will call **`torch.nn.Module.__call__( inputs )`**, which will call **`torch.nn.Module.forward( inputs )`**.

```
bincl_model = BinClassifierMLP( n_input_vars = inputs_train.shape[1] )
train_batch_ys_pred = bincl_model(train_batch_xs)
```

PyTorch - `torch.nn.Module`

```
class BinClassifierMLP(nn.Module):

  def __init__(self, n_input_vars):
        super().__init__()
        self.linear1 = nn.Linear(n_input_vars, 30)
        self.linear2 = nn.Linear(30, 20)
        self.linear3 = nn.Linear(20, 1)

  def forward(self, x):
      h = self.linear1(x)
      h = nn.functional.relu(h)
      h = self.linear2(h)
      h = nn.functional.relu(h)
      h = self.linear3(h)
      h = nn.functional.sigmoid(h)
      return h
```



PyTorch - `torch.nn.Module`

```
class BinClassifierMLP(nn.Module):

  def __init__(self, n_input_vars):
        super().__init__()
        self.linear1 = nn.Linear(n_input_vars, 30)
        self.linear2 = nn.Linear(30, 20)
        self.linear3 = nn.Linear(20, 1)

  def forward(self, x):
      h = self.linear1(x)
      h = nn.functional.relu(h)
      h = self.linear2(h)
      h = nn.functional.relu(h)
      h = self.linear3(h)
      h = nn.functional.sigmoid(h)
      return h
```

We will evaluate the network with **batches** of inputs at once. Therefore, our **input tensors will have a 2D shape**.

Given `x.shape == (batch_size, n_input_vars)`:

PyTorch - `torch.nn.Module`

```
class BinClassifierMLP(nn.Module):

  def __init__(self, n_input_vars):
        super().__init__()
        self.linear1 = nn.Linear(n_input_vars, 30)
        self.linear2 = nn.Linear(30, 20)
        self.linear3 = nn.Linear(20, 1)

  def forward(self, x):
      h = self.linear1(x)
      h = nn.functional.relu(h)
      h = self.linear2(h)
      h = nn.functional.relu(h)
      h = self.linear3(h)
      h = nn.functional.sigmoid(h)
      return h
```

Given `x.shape == (batch_size, n_input_vars)`:

`h.shape == (batch_size, 30)`

PyTorch - `torch.nn.Module`

```
class BinClassifierMLP(nn.Module):

  def __init__(self, n_input_vars):
        super().__init__()
        self.linear1 = nn.Linear(n_input_vars, 30)
        self.linear2 = nn.Linear(30, 20)
        self.linear3 = nn.Linear(20, 1)

  def forward(self, x):
      h = self.linear1(x)
      h = nn.functional.relu(h)
      h = self.linear2(h)
      h = nn.functional.relu(h)
      h = self.linear3(h)
      h = nn.functional.sigmoid(h)
      return h
```

Given `x.shape == (batch_size, n_input_vars)`:

`h.shape == (batch_size, 30)`

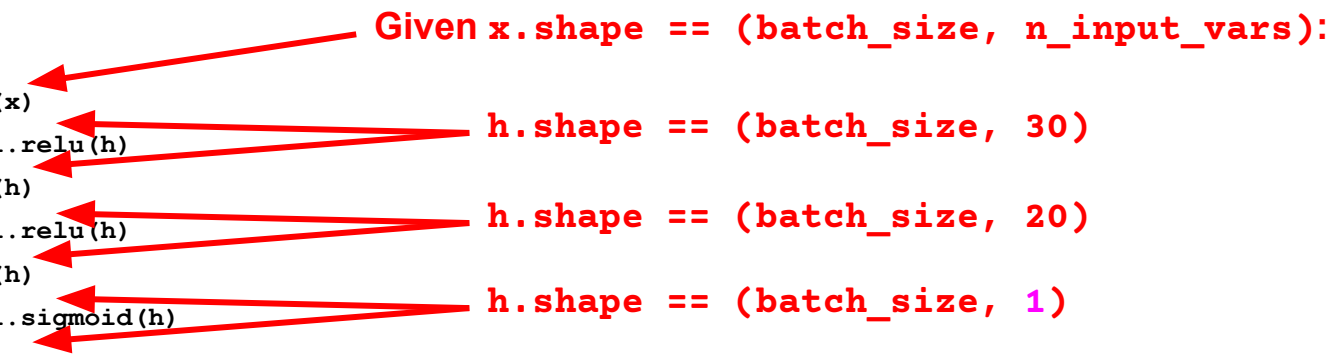
`h.shape == (batch_size, 20)`

PyTorch - `torch.nn.Module`

```
class BinClassifierMLP(nn.Module):

  def __init__(self, n_input_vars):
        super().__init__()
        self.linear1 = nn.Linear(n_input_vars, 30)
        self.linear2 = nn.Linear(30, 20)
        self.linear3 = nn.Linear(20, 1)

  def forward(self, x):
      h = self.linear1(x)
      h = nn.functional.relu(h)
      h = self.linear2(h)
      h = nn.functional.relu(h)
      h = self.linear3(h)
      h = nn.functional.sigmoid(h)
      return h
```



PyTorch - `torch.nn.Module`

```
class BinClassifierMLP(nn.Module):

  def __init__(self, n_input_vars):
        super().__init__()
        self.linear1 = nn.Linear(n_input_vars, 30)
        self.linear2 = nn.Linear(30, 42)
        self.linear3 = nn.Linear(99, 1)

  def forward(self, x):
      h = self.linear1(x)
      h = nn.functional.relu(h)
      h = self.linear2(h)
      h = nn.functional.relu(h)
      h = self.linear3(h)
      h = nn.functional.sigmoid(h)
      return h
```

An example for a shape mismatch:
Layer #2 and Layer #3 has an incompatible number of inputs / neurons specified.

PyTorch - `torch.nn.Module`

```python
class BinClassifierMLP(nn.Module):

  def __init__(self, n_input_vars):
        super().__init__()
        self.linear1 = nn.Linear(n_input_vars, 30)
        self.linear2 = nn.Linear(30, 42)
        self.linear3 = nn.Linear(99, 1)

  def forward(self, x):
      h = self.linear1(x)
      h = nn.functional.relu(h)
      h = self.linear2(h)
      h = nn.functional.relu(h)
      h = self.linear3(h)
      h = nn.functional.sigmoid(h)
      return h
```

An example for a shape mismatch:
Layer #2 and Layer #3 has an incompatible number of inputs / neurons specified.

Error when evaluating the model:

```
/usr/local/lib/python3.13/dist-packages/torch/nn/modules/linear.py in forward(self, input)
    132         Runs the forward pass.
    133         """
--> 134         return F.linear(input, self.weight, self.bias)
    135
    136     def extra_repr(self) -> str:

RuntimeError: mat1 and mat2 shapes cannot be multiplied (32x42 and 99x1)
```

PyTorch - `torch.nn.Module`

```python
class BinClassifierMLP(nn.Module):

  def __init__(self, n_input_vars):
        super().__init__()
        self.linear1 = nn.Linear(n_input_vars, 30)
        self.linear2 = nn.Linear(30, 42)
        self.linear3 = nn.Linear(99, 1)

  def forward(self, x):
      h = self.linear1(x)
      h = nn.functional.relu(h)
      h = self.linear2(h)
      h = nn.functional.relu(h)
      h = self.linear3(h)
      h = nn.functional.sigmoid(h)
      return h
```

An example for a shape mismatch:
Layer #2 and Layer #3 has an incompatible number of inputs / neurons specified.

Error when evaluating the model:

```
/usr/local/lib/python3.13/dist-packages/torch/nn/modules/linear.py in forward(self, input)
    132         Runs the forward pass.
    133         """
--> 134         return F.linear(input, self.weight, self.bias)
    135
    136     def extra_repr(self) -> str:

RuntimeError: mat1 and mat2 shapes cannot be multiplied (32x42 and 99x1)
```

inputs to Layer #3: (batch_size=32) × 42

W matrix of Layer #3: 99 x 1

PyTorch - `torch.nn.Module`

```
class BinClassifierMLP(nn.Module):

  def __init__(self, n_input_vars):
        super().__init__()
        self.linear1 = nn.Linear(n_input_vars, 30)
        self.relu1 = nn.ReLU()
        self.linear2 = nn.Linear(30, 20)
        self.relu2 = nn.ReLU()
        self.linear3 = nn.Linear(20, 1)
        self.sigmoid = nn.Sigmoid()

  def forward(self, x):
      h = self.linear1(x)
      h = self.relu1(h)
      h = self.linear2(h)
      h = self.relu2(h)
      h = self.linear3(h)
      h = self.sigmoid(h)
      return h
```

An alternative approach:
A lot of popular neural network components are implemented as **`torch.nn.Module`** classes too. Calling, for example, an instance of **`torch.nn.ReLU`** will simply call the ReLU function.

However, **it is not necessary to do it this way, as a ReLU activation function is stateless**, so no need to maintain an instance for this wrapper class.

PyTorch - `torch.nn.Module`

```
class BinClassifierMLP(nn.Module):

  def __init__(self, n_input_vars):
        super().__init__()
        self.layers = [
            nn.Linear(n_input_vars, 30),
            nn.ReLU(),
            nn.Linear(30, 20),
            nn.ReLU(),
            nn.Linear(20, 1),
            nn.Sigmoid()
        ]

  def forward(self, x):
    h = x
    for layer in self.layers:
      h = layer(h)
    return h
```

Placing the layers in a list, so when modifying the architecture no need to edit `forward()`:

This will not work correctly !!!
The **`torch.nn.Module`** class will only browse through its members which has the type **`torch.nn.Module`** (or `torch.nn.parameter.Parameter`) to look for parameters that can be trained.
A list is not such an object so it will be skipped!

PyTorch - `torch.nn.Module`

```
class BinClassifierMLP(nn.Module):

  def __init__(self, n_input_vars):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(n_input_vars, 30),
            nn.ReLU(),
            nn.Linear(30, 20),
            nn.ReLU(),
            nn.Linear(20, 1),
            nn.Sigmoid()
        )

  def forward(self, x):
    return self.layers(x)
```

Placing the layers in a `torch.nn.Sequential`, so when modifying the architecture no need to edit `forward()`:

This is the correct way.
The **`torch.nn.Sequential`** class is basically a list implementation that holds other **`torch.nn.Module`** objects (for example layers).
As **`torch.nn.Sequential`** itself is a subclass of **`torch.nn.Module`**, its elements will be searched for trainable parameters too.

Calling the **`Sequential`** instance will call its elements one by one.

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for offset_idx in range(0, inputs_train_t.shape[0], batch_size):
        end_offset_idx = min(offset_idx + batch_size, inputs_train_t.shape[0])
        train_batch_xs = inputs_train_t[offset_idx:end_offset_idx, :]
        train_batch_ys = labels_train_t[offset_idx:end_offset_idx, None]
        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())
```

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for offset_idx in range(0, inputs_train_t.shape[0], batch_size):
        end_offset_idx = min(offset_idx + batch_size, inputs_train_t.shape[0])
        train_batch_xs = inputs_train_t[offset_idx:end_offset_idx, :]
        train_batch_ys = labels_train_t[offset_idx:end_offset_idx, None]
        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())
```

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for offset_idx in range(0, inputs_train_t.shape[0], batch_size):
        end_offset_idx = min(offset_idx + batch_size, inputs_train_t.shape[0])
        train_batch_xs = inputs_train_t[offset_idx:end_offset_idx, :]
        train_batch_ys = labels_train_t[offset_idx:end_offset_idx, None]
        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())
```

`loss_fn:` Specifying the loss function.

`optimizer:` An object encapsulating an optimizer algorithm.

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for offset_idx in range(0, inputs_train_t.shape[0], batch_size):
        end_offset_idx = min(offset_idx + batch_size, inputs_train_t.shape[0])
        train_batch_xs = inputs_train_t[offset_idx:end_offset_idx, :]
        train_batch_ys = labels_train_t[offset_idx:end_offset_idx, None]
        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())
```

`loss_fn:` Specifying the loss function.

`optimizer:` An object encapsulating an optimizer algorithm.

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for offset_idx in range(0, inputs_train_t.shape[0], batch_size):
        end_offset_idx = min(offset_idx + batch_size, inputs_train_t.shape[0])
        train_batch_xs = inputs_train_t[offset_idx:end_offset_idx, :]
        train_batch_ys = labels_train_t[offset_idx:end_offset_idx, None]
        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())
```

`loss_fn`: Specifying the loss function (also a **`torch.nn.Module`** subclass). We can use one of the built-in loss functions…

`optimizer`: An object encapsulating an optimizer algorithm (The SGD variant “Adam”).

The implementation of the (training) **iteration of gradient descent.**

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for offset_idx in range(0, inputs_train_t.shape[0], batch_size):
        end_offset_idx = min(offset_idx + batch_size, inputs_train_t.shape[0])
        train_batch_xs = inputs_train_t[offset_idx:end_offset_idx, :]
        train_batch_ys = labels_train_t[offset_idx:end_offset_idx, None]
        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())
```

The implementation of the (training) **iteration of gradient descent.**

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for offset_idx in range(0, inputs_train_t.shape[0], batch_size):
        end_offset_idx = min(offset_idx + batch_size, inputs_train_t.shape[0])
        train_batch_xs = inputs_train_t[offset_idx:end_offset_idx, :]
        train_batch_ys = labels_train_t[offset_idx:end_offset_idx, None]
        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())
```

A simple way to implement mini-batching: Slice the training set up to batches of tensors shaped `(batch_size, n_input_vars)`.

`n_epochs`: Going through the training set once will not necessarily be enough. We have to iterate until the neural network is trained properly. **One epoch** (usually) is defined as iterating through the training set **once**.

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for offset_idx in range(0, inputs_train_t.shape[0], batch_size):
        end_offset_idx = min(offset_idx + batch_size, inputs_train_t.shape[0])
        train_batch_xs = inputs_train_t[offset_idx:end_offset_idx, :]
        train_batch_ys = labels_train_t[offset_idx:end_offset_idx, None]
        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())
```

Setting the (neural network) **model into training mode.**

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for offset_idx in range(0, inputs_train_t.shape[0], batch_size):
        end_offset_idx = min(offset_idx + batch_size, inputs_train_t.shape[0])
        train_batch_xs = inputs_train_t[offset_idx:end_offset_idx, :]
        train_batch_ys = labels_train_t[offset_idx:end_offset_idx, None]
        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())
```

Setting the (neural network) **model into training mode.** This simply sets the boolean flag:

`nn.Module.training = True`

for the model and all its children **`nn.Module`** objects (e.g., layers) recursively.

Its use will be explained later…

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for offset_idx in range(0, inputs_train_t.shape[0], batch_size):
        end_offset_idx = min(offset_idx + batch_size, inputs_train_t.shape[0])
        train_batch_xs = inputs_train_t[offset_idx:end_offset_idx, :]
        train_batch_ys = labels_train_t[offset_idx:end_offset_idx, None]
        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())
```

Extracting a mini-batch from the input and label tensors (into views) **by slicing.**

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for offset_idx in range(0, inputs_train_t.shape[0], batch_size):
        end_offset_idx = min(offset_idx + batch_size, inputs_train_t.shape[0])
        train_batch_xs = inputs_train_t[offset_idx:end_offset_idx, :]
        train_batch_ys = labels_train_t[offset_idx:end_offset_idx, None]
        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())
```

Extracting a mini-batch from the input and label tensors (into views) **by slicing.**

This serves two purposes:

- **Faster computation** of the predictions, the loss and the gradients over the mini-batch instead of the entire training set tensors.
- **Lower memory usage.**

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for offset_idx in range(0, inputs_train_t.shape[0], batch_size):
        end_offset_idx = min(offset_idx + batch_size, inputs_train_t.shape[0])
        train_batch_xs = inputs_train_t[offset_idx:end_offset_idx, :]
        train_batch_ys = labels_train_t[offset_idx:end_offset_idx, None]
        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())
```

Computes the model predictions.

Calls the loss function.

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for offset_idx in range(0, inputs_train_t.shape[0], batch_size):
        end_offset_idx = min(offset_idx + batch_size, inputs_train_t.shape[0])
        train_batch_xs = inputs_train_t[offset_idx:end_offset_idx, :]
        train_batch_ys = labels_train_t[offset_idx:end_offset_idx, None]
        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())
```

Computes the model predictions by passing the input mini-batch tensor to the **`bincl_model.__call__()`** and thus to **`bincl_model.forward()`**. This is the evaluation of the hypothesis function.

Calls the loss function, which compares the model predictions to the true labels for the current mini-batch and computes the loss value.

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for offset_idx in range(0, inputs_train_t.shape[0], batch_size):
        end_offset_idx = min(offset_idx + batch_size, inputs_train_t.shape[0])
        train_batch_xs = inputs_train_t[offset_idx:end_offset_idx, :]
        train_batch_ys = labels_train_t[offset_idx:end_offset_idx, None]
        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())
```

Performs the backpropagation step.

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for offset_idx in range(0, inputs_train_t.shape[0], batch_size):
        end_offset_idx = min(offset_idx + batch_size, inputs_train_t.shape[0])
        train_batch_xs = inputs_train_t[offset_idx:end_offset_idx, :]
        train_batch_ys = labels_train_t[offset_idx:end_offset_idx, None]
        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())
```

Performs the backpropagation step, starting from the loss expression, going backwards until all model parameters are reached. Fills in the `.grad` tensors for all parameters in the network.

Details of this mechanism will be explained later.

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for offset_idx in range(0, inputs_train_t.shape[0], batch_size):
        end_offset_idx = min(offset_idx + batch_size, inputs_train_t.shape[0])
        train_batch_xs = inputs_train_t[offset_idx:end_offset_idx, :]
        train_batch_ys = labels_train_t[offset_idx:end_offset_idx, None]
        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())
```

Needed for the correct operation of the SGD-based optimizer algorithm.

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for offset_idx in range(0, inputs_train_t.shape[0], batch_size):
        end_offset_idx = min(offset_idx + batch_size, inputs_train_t.shape[0])
        train_batch_xs = inputs_train_t[offset_idx:end_offset_idx, :]
        train_batch_ys = labels_train_t[offset_idx:end_offset_idx, None]
        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())
```

Needed for the correct operation of the SGD-based optimizer algorithm.

`optimizer.zero_grad()`: Needs to be called before each training iteration. This clears the `.grad` tensors of the optimized parameters in the network from the previous iteration.

`optimizer.step()`: Takes the freshly calculated `.grad` tensors for the parameters and applies the gradient update rule to update each parameter by their corresponding gradients scaled with a learning rate.

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for offset_idx in range(0, inputs_train_t.shape[0], batch_size):
        end_offset_idx = min(offset_idx + batch_size, inputs_train_t.shape[0])
        train_batch_xs = inputs_train_t[offset_idx:end_offset_idx, :]
        train_batch_ys = labels_train_t[offset_idx:end_offset_idx, None]
        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())
```

Recording the training loss values for the current epoch.

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    for offset_idx in range(0, inputs_train_t.shape[0], batch_size):
        end_offset_idx = min(offset_idx + batch_size, inputs_train_t.shape[0])
        train_batch_xs = inputs_train_t[offset_idx:end_offset_idx, :]
        train_batch_ys = labels_train_t[offset_idx:end_offset_idx, None]
        optimizer.zero_grad()
        train_batch_ys_pred = bincl_model(train_batch_xs)
        loss = loss_fn(train_batch_ys_pred, train_batch_ys)

        train_losses_in_epoch.append(loss)

        loss.backward()
        optimizer.step()

    train_losses.append(torch.tensor(train_losses_in_epoch).mean())
```

The average of the per-batch loss values will provide the loss value for the current epoch. **These values can be collected and later plotted on a graph to visualize the loss curves.**

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    ...

    bincl_model.eval()

    for offset_idx in range(0, labels_val_t.shape[0], batch_size):
       ...

        with torch.no_grad():
            val_batch_ys_pred = bincl_model(val_batch_xs)
        loss = loss_fn(val_batch_ys_pred, val_batch_ys)

        val_losses_in_epoch.append(loss)

    val_losses.append(torch.tensor(val_losses_in_epoch).mean())
```

PyTorch - A simple training loop

In each epoch:

- Going through the **training set** once by mini-batch iteration and **training**.
- Going through the **validation set** once by mini-batch iteration and **evaluating**.

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    ...

    bincl_model.eval()

    for offset_idx in range(0, labels_val_t.shape[0], batch_size):
       ...

        with torch.no_grad():
            val_batch_ys_pred = bincl_model(val_batch_xs)
        loss = loss_fn(val_batch_ys_pred, val_batch_ys)

        val_losses_in_epoch.append(loss)

    val_losses.append(torch.tensor(val_losses_in_epoch).mean())
```

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    ...

    bincl_model.eval()

    for offset_idx in range(0, labels_val_t.shape[0], batch_size):
       ...

        with torch.no_grad():
            val_batch_ys_pred = bincl_model(val_batch_xs)
        loss = loss_fn(val_batch_ys_pred, val_batch_ys)

        val_losses_in_epoch.append(loss)

    val_losses.append(torch.tensor(val_losses_in_epoch).mean())
```

Setting the (neural network) **model into evaluation mode.**

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    ...

    bincl_model.eval()

    for offset_idx in range(0, labels_val_t.shape[0], batch_size):
       ...

        with torch.no_grad():
            val_batch_ys_pred = bincl_model(val_batch_xs)
        loss = loss_fn(val_batch_ys_pred, val_batch_ys)

        val_losses_in_epoch.append(loss)

    val_losses.append(torch.tensor(val_losses_in_epoch).mean())
```

Setting the (neural network) **model into evaluation mode.**
This simply sets the boolean flag:

`nn.Module.training = False`

for the model and all its children **`nn.Module`** objects (e.g., layers) recursively.

Its use will be explained later…

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    ...

    bincl_model.eval()

    for offset_idx in range(0, labels_val_t.shape[0], batch_size):
       ...

        with torch.no_grad():
            val_batch_ys_pred = bincl_model(val_batch_xs)
        loss = loss_fn(val_batch_ys_pred, val_batch_ys)

        val_losses_in_epoch.append(loss)

    val_losses.append(torch.tensor(val_losses_in_epoch).mean())
```

The implementation of the (validation) **mini-batch iteration.**

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    ...

    bincl_model.eval()

    for offset_idx in range(0, labels_val_t.shape[0], batch_size):
       ...

        with torch.no_grad():
            val_batch_ys_pred = bincl_model(val_batch_xs)
        loss = loss_fn(val_batch_ys_pred, val_batch_ys)

        val_losses_in_epoch.append(loss)

    val_losses.append(torch.tensor(val_losses_in_epoch).mean())
```

The implementation of the (validation) **mini-batch iteration.**

As we are not training, the only use of this is to reduce the memory usage.

Generally, we should aim to use as much memory at once as we can (especially when using a GPU). If the evaluation of the model over the entire validation set fits into the memory, there is no need for mini-batching here.

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    ...

    bincl_model.eval()

    for offset_idx in range(0, labels_val_t.shape[0], batch_size):
       ...

        with torch.no_grad():
            val_batch_ys_pred = bincl_model(val_batch_xs)
        loss = loss_fn(val_batch_ys_pred, val_batch_ys)

        val_losses_in_epoch.append(loss)

    val_losses.append(torch.tensor(val_losses_in_epoch).mean())
```

Computes the model predictions.

Calls the loss function.

The loss is only used here for evaluation purposes.

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    ...

    bincl_model.eval()

    for offset_idx in range(0, labels_val_t.shape[0], batch_size):
       ...

        with torch.no_grad():
            val_batch_ys_pred = bincl_model(val_batch_xs)
        loss = loss_fn(val_batch_ys_pred, val_batch_ys)

        val_losses_in_epoch.append(loss)

    val_losses.append(torch.tensor(val_losses_in_epoch).mean())
```

Recording the validation loss values for the current epoch, which can be plotted…

PyTorch - A simple training loop

```
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(bincl_model.parameters(), lr=learning_rate)

for epoch_idx in range(n_epochs):
    bincl_model.train()

    ...

    bincl_model.eval()

    for offset_idx in range(0, labels_val_t.shape[0], batch_size):
       ...

        with torch.no_grad():
            val_batch_ys_pred = bincl_model(val_batch_xs)
        loss = loss_fn(val_batch_ys_pred, val_batch_ys)

        val_losses_in_epoch.append(loss)

    val_losses.append(torch.tensor(val_losses_in_epoch).mean())
```

No need to create computational graphs and compute gradients inside this block as we are not training here.

(Will be discussed later, in detail.)

PyTorch - A simple training loop

Optionally, additional features can be included in the training loop:

- We can **evaluate** the performance of the network with **other metrics** too (e.g., accuracy for classification).
- After we finished the training procedure (outside of the external loop), we can **evaluate** the performance of the network **on the test set.**
- We can include **early stopping** *(later).*
- …